

개발 기록

저장소 `nfinity/README.md` 의 내용이다. 개발 중 발견한 문제와 그 원인, 판단 근거가 시간순으로 기록되어 있다.

Nfinity - 백엔드 (Day 1~4 병합본 + 검증 완료, 8/28 기준)

전공 지식이 아직 부족해도 따라할 수 있게, 개념부터 실행법까지 최대한 쉽게 적었습니다.

이 README는 은겸님이 세 분(본인/소현/하은)이 각자 만든 산출물을 하나로 합치고, 실제 PostgreSQL을 띄워서 전체 파이프라인을 끝까지 돌려본 뒤 정리한 버전입니다. (아래 "4. 실제로 검증한 것" 참고) 8/26 백엔드 구축 데이(6-1), 8/28 AI 절세장부 실연동(6-2), 8/28 Isolation Forest 이상탐지(6-3)까지 전부 반영해서 다시 검증했습니다.

1. 지금 이게 뭔가요?

세 가지 프로그램이 서로 연결돼서 돌아갑니다.

- **PostgreSQL** — 데이터를 저장하는 곳. 유저, 거래 내역, 이상 거래 기록이 여기 저장됩니다.
- **Redis** — 아주 빠르게 읽고 쓰는 임시 저장소. 예산통제 기능에서 "이번 달 카테고리별로 얼마 썼는지"를 실시간으로 집계할 때 씁니다. (8/26부터 실제로 사용 시작 — `app/redis_client.py`)
- **FastAPI** — 우리가 만드는 서버. "이 거래 위험한가요?" 같은 질문을 받아서(API) 답을 주는 프로그램입니다.

Docker는 이 세 프로그램을 각자의 컴퓨터에 굳이 설치 안 해도, 명령어 한 줄로 통째로 띄워주는 도구입니다.

2. 폴더 구조

```
sidegig_ai/
├── docker-compose.yml    # 3개 프로그램(DB, Redis, API) 실행 설명서
├── Dockerfile
├── requirements.txt
├── .env / .env.example
├── sql/
│   └── init.sql          # DB 테이블 설계도
├── app/
│   ├── main.py           # 라우터 등록 + 공통 에러 응답 포맷 통일 (★8/26 추가)
│   ├── models.py
│   ├── database.py
│   ├── redis_client.py   # ★신규(8/26): Redis 실제 연동 헬퍼 (예산 실시간 집계용)
│   ├── rule_engine.py    # 리스크 판단 규칙 7개 (R001~R007)
│   └── routers/
│       ├── risk.py       # "/api/v1/risk/..." (severity/action 집계 버그 수정됨 - 4번 참고)
│       ├── budgets.py    # ★신규(8/26): "/api/v1/budgets/..." 예산통제
│       ├── gig_score.py  # ★신규(8/26): "/api/v1/gig-score/{user_id}" (임시 산식)
│       └── expense.py     # ★신규(8/26): "/api/v1/expense/classify" - 하은님이 채울 자리
├── data_pipeline/        # ★신규: 하은님 파이프라인 (기존 위치에서 이동 + import 경로 수정)
│   ├── cleanser.py
│   ├── categorizer.py
│   └── ingestion.py
├── data/                 # ★신규: 소현님 mock 데이터
│   ├── mock_transactions.csv      (4,978건, 35명)
│   └── mock_transactions_answer_key.csv
```

```
| | persona_map.csv
| | seed_data.py
| | scripts/
| | build_user_profiles.py # ★신규: Day 3 작업 - 실제 거래 기반 프로파일 계산
| | tests/
| | test_rule_engine.py
```

3. 실행 방법 (본인 컴퓨터에 Docker Desktop 필요)

```
cd sidegig_ai
cp .env.example .env
docker compose up --build -d
```

30초~1분 후 브라우저에서:

```
http://localhost:8000/docs
```

DB에 실제 데이터를 채워 넣으려면 (컨테이너가 뜬 상태에서):

```
pip install -r requirements.txt
export DATABASE_URL=postgresql://sidegig:sidegig_pw@localhost:5432/sidegig
python -m data_pipeline.ingestion # mock_transactions.csv 4,978건 적재 (정제+카테고리분류 포함)
python -m scripts.build_user_profiles # 유저별 실제 한도/평균 프로파일 계산
```

컨테이너 상태 확인: `docker compose ps` · 로그: `docker compose logs -f api` · 끄기: `docker compose down`

4. 실제로 검증한 것 (Docker 없이, 로컬 PostgreSQL 16으로 동일 코드 실행)

이 환경엔 Docker 데몬이 없어서 `docker compose up` 은 직접 못 돌렸지만, **완전히 동일한 코드**로 로컬 PostgreSQL을 띄워 Day1(A)+Day2(B,C) 산출물을 전부 병합해서 처음부터 끝까지 실행했습니다.

`docker-compose.yml` / `Dockerfile` 은 표준 구성이라 본인 컴퓨터에서도 그대로 동작할 거예요.

- ✓ `sql/init.sql` → 6개 테이블 정상 생성
- ✓ `data_pipeline/ingestion.py` → `mock_transactions.csv` **4,978건**(`persona_map.csv` 기준 유저 35명) 전부 정제 → 카테고리분류 → 적재 성공. 가맹점 23종이 `categorizer.py` 매핑에 전부 커버되어 "기타"로 새는 거래 0건.
- ✓ `scripts/build_user_profiles.py` (신규) → 35명 전원 실제 거래 기반 프로파일 계산 완료
- ✓ `tests/test_rule_engine.py` 6개 테스트 전부 통과
- ✓ `POST /api/v1/risk/assess` 실서버 기동 후 실제 HTTP 요청으로 확인
- 🐛 **버그 발견 & 수정:** `app/routers/risk.py` 에서 여러 규칙이 동시에 걸렸을 때 최종 `severity / action` 을 뽑는 로직이 `max(results, key=lambda r: r.severity)` 로 **Severity를 알파벳 문자열 순으로 비교**하고 있었습니다. 그 순서가 `CRITICAL < HIGH < LOW < MEDIUM` 이라, 예를 들어 R001(HIGH)과 R002(CRITICAL)가 동시에 걸려도 최종 결과가 `MEDIUM / HIGH` 처럼 실제보다 낮게 나올 수 있었습니다 (심야에 한도초과+해외결제가 겹치는 등 흔히 일어나는 상황).
→ 심각도 랭킹표(`_SEVERITY_RANK`)로 교체해서 수정했고, 실제 API 응답으로 재확인했습니다 (한도 초과 상황 테스트 → 이제 정확히 `CRITICAL / BLOCK` 반환). `GET /risk/score/{user_id}` 의 동시각 타이브레이크 문제도 같은 방식으로 같이 고쳤습니다.
- 🎨 **룰 엔진만으로(ML 모델 불이기 전)** mock 데이터 정답지 대비 정확도: **Precision 0.61 / Recall 0.65**. 낮아 보이지만 정상입니다 — 정답지의 이상 유형 중 `R00y_신규기기` (기기 불일치)와 `R00x_위치급변` 은 애초에 설계서상 **Isolation Forest** 뛴(rule 아님)이고, `R006_단기반복` 은 3연타 이벤트 중

처음 2건은 "아직 반복이 안 쌓인 시점"이라 규칙상 당연히 안 걸립니다(3번째부터 걸림 — 정상 동작).
→ (8/28 업데이트) 실제로 Isolation Forest를 붙이고 재검증한 결과는 아래 6-3번 참고 —
Recall은 0.65→0.734로 올랐고, 그 과정에서 발견한 다른 이슈도 같이 정리했습니다.

5. 팀원별로 알아야 할 것

- A(은겸)** — `app/`, `sql/`, `scripts/build_user_profiles.py` 담당. Day3은 Isolation Forest를 `app/anomaly_model.py` 로 분리해서 룰 엔진 점수와 앙상블하는 것.
- B(소현)** — mock 데이터는 확인 완료(4,978건, 컬럼 계약 100% 일치). 실제 건수는 "14,000건"이 아니라 **4,978건**이니 발표자료에 쓸 때 숫자만 정정해주세요.
- C(하은)** — `cleanser.py` / `categorizer.py` / `ingestion.py` 세 개 다 실제 DB에 붙여서 정상 동작 확인했습니다. `ingestion.py` 는 import 경로만 `data_pipeline`. 패키지 기준으로 살짝 고쳤고, 재실행해도 안전하도록(중복 적재 방지) `TRUNCATE` 옵션을 추가했습니다.

6-1. 8/26 백엔드 구축 데이 — 새로 생긴 API들

원래 계획대로면 오늘부터 Isolation Forest 모델링을 시작할 차례였지만, 은겸님 요청으로 하루를 먼저 "백엔드 뼈대 다지기"에 썼습니다. 목적은 **소현님(프론트)과 하은님(절세장부 LLM)이 오늘부터 안정된 API 위에서 작업을 시작할 수 있게** 하는 것입니다.

무엇을 만들었나

엔드포인트	메서드	상태	설명
<code>/api/v1/budgets</code>	POST	✅ 실동작	카테고리별 예산 한도 설정
<code>/api/v1/budgets/usage</code>	GET	✅ 실동작	이번 달 카테고리별 사용량 (Redis 실시간 집계)
<code>/api/v1/budgets/record-spending</code>	POST	✅ 실동작 (임시)	거래 1건만큼 누적 지출에 더하기. 지금은 수동 호출용, 나중에 카테고리분류 파이프라인이 자동 호출하도록 연결 예정
<code>/api/v1/budgets/forecast</code>	GET	✅ 실동작 (단순버전)	월말 예상 지출 — "지금까지 쓴 돈/지난 날짜수 × 이번달 날짜수"의 가장 단순한 선형 예측. 정기결제 반영 등은 8/30-31에 추가
<code>/api/v1/gig-score/{user_id}</code>	GET	⚠️ 임시 산식	거래 빈도(activity)+지출 안정성(stability) 두 가지만 보는 자리표시자. 세금 파킹 성실도 등 실제 지표는 8/30-31에 추가
<code>/api/v1/expense/classify</code>	GET	✅ 실동작 (8/28)	조소현님이 완성한 Gemini 프롬프트 로직과 실제 연동. 아래 6-2 참고

실제로 검증한 것

로컬 PostgreSQL + Redis를 띄우고 7개 시나리오(예산 설정 → 지출 2건 기록 → 사용량 조회 → 예측 조회 → Gig-Score 조회 → 절세장부 스텝 호출 → 존재하지 않는 유저 에러 확인)를 실제 HTTP 요청으로 전부 통과 확인했습니다.

만들면서 발견해서 고친 것 2개

- budgets 테이블에 UNIQUE 제약이 없었습니다.** 원래 설계서에는 `UNIQUE(user_id, category, period)` 가 있었는데 실제 `sql/init.sql` 에는 빠져 있어서, 예산을 두 번 설정하면

같은 유저·카테고리 행이 계속 늘어나는 상태였습니다. `sql/init.sql` 에 제약을 추가하고 (이미 테이블이 있는 사람을 위한 `ALTER` 보강문도 같이 넣었습니다), `ON CONFLICT ... DO UPDATE` 로 "이미 있으면 갱신"이 되도록 고쳤습니다.

2. `get_usage()` 에서 존재하지 않는 `is_active` 컬럼을 조회 조건에 걸어서 처음엔 에러가 났습니다 (설계서 원본 스키마와 실제 `budgets` 테이블이 달랐음) — 조건을 제거해서 고쳤습니다.

공통 에러 응답 포맷 통일

이전에는 API마다 FastAPI 기본 에러 형식(`{"detail": "...}"`)을 그대로 썼는데, `app/main.py` 에 예외 핸들러를 추가해서 이제 **모든 API가 항상 같은 모양**으로 에러를 반환합니다:

```
{"error": {"code": 404, "message": "유저 프로파일이 없습니다."}}
```

프론트(소현)는 이제 에러 처리 코드를 한 군데만 만들면 됩니다.

Redis, 이제 진짜로 씁니다

`app/redis_client.py` 에 `add_spending()` / `get_spending()` 두 함수를 만들었습니다. 거래 금액을 Redis의 `INCRBYFLOAT` (현재 값에 원자적으로 더하기)로 누적하고, 40일 뒤 자동 만료되게 TTL을 걸어뒀습니다. `GET /health` 에서 Redis 연결 상태도 같이 확인할 수 있습니다.

6-2. 8/28 AI 절세장부 — 조소현님 분류 로직 실연동

조소현님이 `data_pipeline/categorizer.py` 를 확장해서 실제 경비분류 프롬프트(Gemini, few-shot 4개, 규칙 기반 폴백, 하이브리드 스킵 로직)를 완성해 전달해주셨습니다 (`expense_classifier.py` + `sample_expense_classify_response.json`). 이번에 은겸이 이걸 `GET /api/v1/expense/classify?user_id=...&limit=15` 에 실제로 연결했습니다 — DB에서 그 유저의 실제 거래를 읽어와 조소현님 로직(`classify_one()`)에 그대로 넘기고, 합의된 응답 모양으로 포장만 하는 얇은 라우터입니다.

API 계약 (변경 금지 — 프론트 하은님과 이미 합의됨):

```
{
  "transactions_queue": [
    { "id": 1, "merchant": "스타벅스", "amount": 15000, "ai_tag": "업무미팅", "prob": 92 }
  ],
  "current_estimated_refund": 125000
}
```

전달받은 파일에서 고친 것 (로직/프롬프트는 그대로, 실행 환경 버그만 수정):

- 🚩 `call_openai()` 안에 `os.environ.get(AQ.Ab8RN6...)` 처럼 **따옴표 없이 API 키로 보이는 문자열이 코드에 그대로 박혀 있었습니다**. 문법 오류라 실행은 안 됐겠지만, 실제 키를 실수로 커밋한 것처럼 보입니다. 소현님 — 이 키가 진짜라면 지금 바로 재발급받아주세요 (파일이 팀 zip으로 여러 사람 거쳐 공유됐습니다). 이 버전은 `os.environ.get("GEMINI_API_KEY")` 로 정상화했습니다.
- `MOCK_MODE` 가 파일에 두 번(7번째 줄 `False`, 46번째 줄 `True`) 정의되어 있어서 실제로는 항상 Mock으로만 동작했습니다 → `GEMINI_API_KEY` 존재 여부로 자동 결정하도록 정리.
- `run_batch()` 의 CSV 경로가 개인 컴퓨터 절대경로(`C:/Users/...`)로 하드코딩되어 있어서 다른 환경에서 실행이 안 됐습니다 → `data/` 상대경로로 변경.

검증: 로컬 PostgreSQL의 실제 거래 데이터(4,978건)로 `GET /api/v1/expense/classify` 를 호출해서 소현님이 보내주신 `sample_expense_classify_response.json` 과 필드명·값 형태가 정확히 일치하는 것을 확인했습니다. 존재하지 않는 유저 조회 시 표준 에러 포맷(404)도 정상 동작합니다. `Mock_MODE` (=Gemini 키 없을 때)로 확인했고, 실제 Gemini 키를 넣고 호출하는 것까지는 이 환경에 키가 없어 확인하지 못했습니다 — 키가 준비되면 소현님/은경님 중 아무나 `.env` 에 `GEMINI_API_KEY` 만 넣고 다시 호출해보면 됩니다(코드 변경 불필요).

남은 일: 지금은 "최근 N건"을 그때그때 다시 분류합니다. 나중에 결과를 `transactions` 테이블에 컬럼(`expense_prob`, `expense_tag`)으로 캐싱해서 스와이프로 승인/거절한 결과를 저장하는 기능을 붙이면 좋을 것 같습니다 (Tinder UI의 "넘기면 사라짐" 동작을 서버가 기억하게 하는 부분 — 시간 되면 8/30~31에).

6-3. 8/28 Isolation Forest 이상 탐지 모델 (은경 담당, Day 4)

정답지(`data/mock_transactions_answer_key.csv`)를 까보면 이상 거래 263건 중 36건 (`R00x`_위치급변 21건 + `R00y`_신규기기 15건)은 애초에 룰 엔진으로 설계되지 않은 유형이라 `R001~R007`로는 절대 못 잡습니다. 이 두 유형을 잡는 게 목표였습니다.

만든 것: `app/anomaly_model.py` — scikit-learn `IsolationForest` 를 학습해서 `models/anomaly_iforest.pkl` 로 저장하고, `app/routers/risk.py` 의 `/risk/assess` 에서 룰 엔진과 함께 앙상블합니다. 모델 파일은 이미 학습해서 zip에 포함했으니 바로 씁니다 (데이터가 바뀌면 `python -m app.anomaly_model` 로 재학습).

앙상블 방식: `combined_prob = 1 - (1-rule_prob) * (1-anomaly_prob)` (확률적 OR). 룰이 이미 CRITICAL이면 AI 결과와 무관하게 100점 그대로 — 기존 룰 기반 동작은 안 바뀝니다. 반대로 룰이 하나도 안 걸렸는데 AI가 확신하면 (`anomaly_prob >= 0.6`), `rule_id="AI001"` 로 `risk_events` 에 남습니다. 실제 정답지 거래로 재현해보면:
- `R00y`_신규기기 샘플(새 기기로 결제) → 룰은 하나도 안 걸리지만 `AI001`이 `MEDIUM/FLAG_FOR_REVIEW`로 정확히 잡음 (`anomaly_prob=0.75`)
- 정상 거래 샘플 → `LOW/ALLOW` 그대로 유지 (오탐 없음)

피쳐 튜닝 — 처음엔 8개였다가 4개로 줄였습니다. 시간대(hour)/요일(dow) 순환 인코딩까지 넣었더니 오히려 신호가 희석돼서(IsolationForest가 분기마다 피쳐를 무작위로 골라 쓰다 보니 의미 없는 피쳐가 많을수록 진짜 신호가 있는 피쳐가 뿔힐 확률이 줄어듭니다), 타깃 유형 탐지가 8개 피쳐일 땐 6/36건이었는데 `amount_zscore / is_new_device / distance_from_prev_km / merchant_rarity` 4개로 줄이니 20/36건으로 올랐습니다. 시간대 이상은 이미 `R005`(심야 대액)가 맡고 있으니 역할을 나눈 셈입니다.

 **만들면서 발견해서 같이 고친 버그** (이상 탐지 피쳐 계산을 위해 히스토리를 제대로 불러오다가 발견함): `app/routers/risk.py` 의 `_load_recent_history()` 가 `datetime.utcnow() - timedelta(days=7)` 기준이었습니다. 즉 "지금 서버가 실행되는 시각" 기준이었는데, mock 데이터는 2026-05-25~08-22이고 이 서버가 실행되는 "지금"(8월 말)과 비교하면 7일 이내로 들어오는 데이터가 사실상 없어서 **`R003`(신규가맹점)/`R004`(불가능한 이동속도)/`R006`(단기반복)**이 실제 DB 데이터로는 거의 항상 빈 이력을 받아 사실상 죽어있는 상태였습니다. 평가 대상 거래 "자신의 시각" 기준으로 바꾸고 윈도우도 180일로 넓혔습니다.

전체 재검증 (룰 엔진 + Isolation Forest, mock 데이터 4,978건 전체, 시간순 재현):

	이전 (룰 엔진만, 히스토리 버그 있는 채로 측정)	지금 (버그 수정 + IF 앙상블)
Recall	0.65	0.734

	이전 (롤 엔진만, 히스토리 버그 있는 채로 측정)	지금 (버그 수정 + IF 양상블)
Precision	0.61	0.555

Recall은 올랐지만 Precision은 떨어졌습니다 — 이유를 추적해보니 **오탐 155건 중 109건이 R002(일일 누적 한도 초과)**에서 나왔습니다. 히스토리 버그가 고쳐지면서 R002가 (전에는 거의 항상 비활성 상태였다가) 이제 실제로 하루 누적액을 제대로 계산하게 됐는데, Day 3에서 만든 `daily_limit` 산식($\text{평균거래액} \times \text{일평균거래건수} \times 3$)이 생각보다 타이트해서 정상적인 씬스임도 자주 걸립니다. **이건 새 버그가 아니라 원래도 있었는데 버그 때문에 가려져 있던 문제입니다.** 지금 당장 고치진 않았고, `daily_limit` 배수를 완화하는 걸 다음 작업으로 남겨둡니다 (아래 7번 참고). AI 모델 자체의 오탐은 46건(정상 거래 4,715건 중 1.0%)으로 낮은 편입니다.

6-4. 8/30 정기결제 감지 + 예산 알림 임계값 (조소현님 로직 실연동)

소현님이 `data_pipeline/recurring_detector.py` 를 새로 만들어서 전달해주셨습니다. 같은 유저+가맹점+비슷한 금액($\pm 5\%$)이 대략 30일 간격으로 반복되면 "정기결제"로 판단하는 클러스터링 로직인데, 실제 mock 데이터로 돌려보니 11건(넷플릭스류 구독, 카카오택시 등)이 잡혔고 합성 테스트(넷플릭스 패턴은 잡고 불규칙한 스타벅스는 거름)도 통과했습니다.

연동한 것:

- GET `/api/v1/budgets/forecast` — 기존 "지금까지 지출 속도 $\times$ 남은 날"(`base_predicted_total`)에 더해서, 이 카테고리에 속하는 가맹점의 정기결제 중 "이번 달 안에 아직 안 지나간 것"의 예상 금액을 `recurring_payments` 필드로 추가하고 `predicted_total` 에 합산했습니다. 가맹점→카테고리 매칭은 `categorizer.py` 의 `categorize_merchant()` 를 그대로 재사용.
- GET `/api/v1/budgets/usage`, POST `/api/v1/budgets` — `crossed_thresholds` 필드 추가 (예: `[50, 80]` = 50%/80%는 이미 넘었고 90% 이상은 아직). `budgets.alert_thresholds` 는 0.5/0.8 같은 "비율"로 저장돼 있어서 소현님 함수가 기대하는 `%(50/80)` 단위로 변환해서 넘겼습니다.

설계 판단: 소현님이 만든 `forecast_month_end()` 는 "지금까지 쓴 돈"까지 자체적으로 다시 계산하는 독립 함수였는데, 이미 검증된 기존 Redis 기반 계산(`get_spending()`)이 있어서 그건 그대로 두고 "정기결제 예상액을 더하는" 부분만 가져와 엮었습니다. 두 군데서 "지금까지 쓴 돈"을 각각 다르게 계산하면 나중에 둘이 어긋날 수 있어서, 소스 오브 트루스를 하나로 유지하기 위한 선택입니다.

실제로 검증한 것: 실제 유저(노션 정기결제 보유)로 예산 생성 → 지출 기록 → `crossed_thresholds` 가 `[50]` 으로 정확히 뜨는 것 확인. 정기결제 카테고리 필터링도 DB에서 직접 불러온 데이터로 검증(노션 → 업무용 카테고리로 정확히 매칭).

⚠ **데모 준비 시 알아둘 것:** `recurring_payments` 는 "이번 달 안"인지를 서버가 실행되는 실제 오늘 날짜(`date.today()`) 기준으로 판단합니다. mock 데이터는 8/22까지라 감지된 정기결제 다음 예정일이 대부분 8월 말~9월 초예요 — 데모를 9월 초 이후에 하면 자연스럽게 채워지고, 그전에 미리 시연하려면 발표 리허설 날짜에 맞는 정기결제가 있는 유저를 골라서 보여주시면 됩니다 (이미 정기결제 실행 로그에 유저별 다음 예정일이 다 나와있어요).

참고: `expense_classifier.py` 도 같이 재전달받았는데, 지난번 지적드린 API 키 노출 버그와 경로 문제를 소현님이 직접 고치신 버전이었습니다 — 저희가 이미 반영한 버전과 로직은 동일해서 기존 것을 그대로 유지했습니다. `.env` 의 `GEMINI_API_KEY` 는 아직 실제 키가 아니라 안내 문구만 있어서, 실제 키가 준비되면 그 줄만 교체하면 됩니다.

6. 룰 엔진 테스트만 빠르게 돌려보는 법 (Docker/DB 없이도 가능)

```
python3 -m venv venv
source venv/bin/activate      # Windows는 venv\Scripts\activate
pip install -r requirements.txt pytest
python -m pytest tests/test_rule_engine.py -v
```

6-5. 8/30 프론트엔드 디자인 시안 + 보안 점검

디자인 시안 (Claude Design 캔버스)

민트-그린 헬스핏 톤으로 예산통제(홈)/리스크방어/절세장부 3개 화면 정적 시안을 만들었습니다 (design/Main.dc.html , design/RiskDefense.dc.html , design/ExpenseLedger.dc.html , design/canvas.json , 시드된 결과물은 design/sidegig-ai-design.html). 캐릭터/마스코트 없이, 실제 API 응답 필드(BudgetUsage , BudgetForecast.recurring_payments , RiskAssessment , ExpenseClassifyResponse 등)에 맞춘 값으로 채워서 백엔드 연동 시 그대로 옮기기 쉽게 만들었습니다. 아직은 정적 시안(하드코딩된 예시 데이터)이고, 실제 API 연동은 다음 단계입니다.

보안 점검 (기존 코드 전체 스캔 + 수정)

발견한 것과 조치한 것:

- **SQL 인젝션:** 전 라우터가 SQLAlchemy text() + 바인드 파라미터(:uid 등)만 쓰고 있어서 문제 없음을 확인했습니다 (문자열 조합으로 쿼리를 만드는 곳 없음).
- **.gitignore 가 아예 없었음 (심각):** .env (API 키/DB 비번)가 그대로 깃허브에 올라갈 뻔했습니다. .gitignore 새로 추가 — .env , __pycache__ , pgdata/ 등 제외. (models/*.pkl 은 처음엔 같이 제외했다가 8/30 리뷰에서 배포 버그로 이어지는 걸 발견해서 다시 포함시켰습니다 — 아래 6-6 "남은 것 / 배포 시 주의" 참고.)
- **모든 API에 인증이 전혀 없었음:** user_id 를 쿼리 파라미터로 그냥 받다 보니, API 주소만 알면 누구나 다른 사람의 예산/리스크/절세장부를 조회할 수 있었습니다. 로그인 시스템은 남은 일정 에 비해 과해서, 대신 app/security.py 에 공유 키(X-API-Key 헤더) 방식의 최소 접근 제어를 추가했습니다. 로컬 개발 중(.env의 API_ACCESS_KEY 가 비어있을 때)은 기존처럼 인증 없이 Swagger UI로 테스트되고, 값을 채우면 모든 /api/v1/* 요청에 그 키가 필수가 됩니다. **깃허브 배포 시 반드시 이 값을 채워야 합니다** (안 그러면 배포된 API가 계속 열려있는 상태) — README 배포 단계에서 다시 강조합니다. 이건 "로그인 전 임시 방어선"이고, user_id를 바꿔치기하는 것 자체를 막는 진짜 인가(authorization)는 아니라는 점은 남겨둡니다.
- **CORS 설정이 아예 없었음:** 프론트를 API와 다른 주소(포트)에서 띄우면 브라우저가 요청을 막습니다. app/main.py 에 CORS를 추가하되 * (전체 허용)는 쓰지 않고, 로컬 개발 주소들만 기본 허용 목록에 넣었습니다. 배포 시엔 .env 의 CORS_ALLOWED_ORIGINS 에 실제 프론트 주소를 추가하세요.
- **docker-compose.yml 에 DB 비밀번호가 하드코딩:** .env 의 POSTGRES_PASSWORD 를 읽도록 바꿨습니다 (기본값은 기존과 동일하게 유지되어 있어서 지금 당장 뭔가 바꿀 필요는 없습니다).
- pickle.load 로 모델 파일을 불러오는 부분(app/anomaly_model.py)이 있는데, 이건 팀이 직접 학습시킨 파일만 읽는 용도로 외부 입력이 아니라서 위험도는 낮다고 판단했습니다(참고용으로 남김).

로컬에서 python -m pytest tests/ 와 실제 uvicorn 서버로 위 4가지(인증 꺼짐/틀린 키/맞는 키/헬스체크)를 직접 요청 보내서 검증했습니다.

6-6. 8/30 프론트-백엔드 실연동 (데모 모드)

왜 "데모 모드"인가

공모전(2026 금융 AI Challenge, 금융보안원 주최·데이콘 운영, daker.ai) 게시판에서 운영진이 직접 "개인화 기능을 실제 데이터 대신 가상(더미) 데이터로 구현·시연해도 된다. 다만 가상 데이터임을 화면에 안내하는 걸 권장한다"고 공식 답변했습니다. 그래서 로그인/개인정보 입력 없이, "미리 만들어둔 인물의 폰을 잠깐 구경하는" 방식으로 갑니다 — 화면 상단에 항상 "실제 인물이 아닌, 시연을 위해 준비한 가상 데이터입니다" 배지를 표시합니다.

무엇을 만들었나

- `app/demo_personas.py` : 스토리가 뚜렷한 5명의 시드 페르소나(김민철·박지수·이하늘·최유진·정다운, 각각 평범(약간의 이상 결제 포함)/자금위기/이상거래 확인 필요/예산초과/N잡 소득변동)를 한 곳에서 관리합니다. (이하늘은 원래 "normal"로 적어뒀었는데, 8/30 검토에서 실제 리스크 점수가 CRITICAL(100점)로 나오는 걸 발견해서 스토리를 실제 결과에 맞게 다시 썼습니다 — 아래 검증 항목 참고.)
- `app/routers/demo.py` : `GET /api/v1/demo/personas` — 프론트 페르소나 선택 화면이 이 API로 카드 5개를 그립니다.
- `scripts/seed_demo_personas.py` : 1회성 시딩 스크립트. **숫자를 지어내지 않습니다** — DB에 이미 있는 이 5명의 실제 mock 거래를 카테고리별로 실제로 합산해서 Redis 예산 카운터에 반영하고(`budgets.py`가 원래 "나중엔 자동 연결" 예정이라고 적어뒀던 부분), 전체 거래를 시간순으로 리스크 평가 로직에 흘려보내 `risk_events`를 채웁니다(이전엔 이 테이블이 비어있어서 리스크 점수가 항상 0이었습니다). 예산 한도(limit)만 "이 페르소나 스토리에 맞게" 정한 값이고, 지출액 자체는 전부 실제 데이터입니다. 재실행해도 안전합니다(idempotent).
- `frontend/` : 정적 HTML/CSS/바닐라JS 데모 웹앱. `index.html` (페르소나 선택) → `budget.html` / `risk.html` / `expense.html` 이 각각 실제 `/api/v1/budgets/*`, `/api/v1/risk/*`, `/api/v1/expense/classify`를 `fetch()`로 직접 호출해서 그립니다. 하드코딩된 예시 데이터는 하나도 없습니다 — `design/`폴더의 정적 시안을 그대로 살아있는 화면으로 바꾼 것입니다.
- `app/main.py`가 API와 프론트를 같은 프로세스·같은 origin에서 함께 서빙합니다(CORS 불필요, 배포용 URL 하나로 끝남 — 공모전 MVP가 "실행 가능한 배포 URL 1개"를 요구하기 때문입니다).

검증

- Playwright로 5개 페르소나 × 3개 화면을 실제로 렌더링해서 스크린샷으로 확인했습니다 (콘솔 에러 없음 — 폰트 CDN 요청 실패는 샌드박스 네트워크 제약 때문이고 폴백 폰트로 정상 표시됩니다).
- 실제로 렌더링해보다가 버그 하나를 발견해서 고쳤습니다: `GET /risk/score/{user_id}`가 `action` 컬럼을 안 내려줘서, 점수가 91(HIGH)인데도 화면엔 "이상 없어요"라고 뜨는 모순이 있었습니다 — SELECT에 `action` 추가해서 해결했습니다.
- `GET /risk/events`에 `transactions` LEFT JOIN을 추가해서 타임라인에 가맹점명/금액이 같이 나오도록 보강했습니다.
- 예산 한도(limit)를 처음엔 대충 잡았다가, 최유진처럼 실지출이 매우 큰 페르소나는 쇼핑 한 카테고리가 5000%대로 튀어서(버그처럼 보임) 실제 지출 규모를 먼저 조회한 뒤 스토리에 맞게 재조정했습니다.

- `python -m pytest tests/` 전체 통과 확인.

남은 것 / 배포 시 주의

- **Dockerfile이 `data_pipeline / models / frontend` 를 안 넣고 있던 버그를 같이 고쳤습니다** — 원래대로 배포했으면 컨테이너가 뜨자마자 `ImportError`로 죽었을 것입니다(6-5번 보안 점검과 별개로 발견한 배포 버그). `--reload` 도 운영 이미지에서 뺐고, 로컬 개발용 `--reload` 는 `docker-compose.yml` 의 `command` 로 옮겼습니다.
- 배포(6단계) 시 `.env` 의 `API_ACCESS_KEY` 를 채우면, `frontend/api.js` 의 `API_KEY` 상수도 같은 값으로 채워서 다시 배포해야 프론트가 API를 계속 호출할 수 있습니다(안 그러면 프론트도 401을 받습니다).

8/30 연동 이후 10회 자체 검토에서 추가로 찾은 것

프론트-백엔드 연동을 끝낸 뒤, 사용자 요청으로 오류/개선점을 단계별로 10번 검토했습니다.

새로 고친 것들:

- **XSS 방어 누락:** 4개 프론트 화면 전부 API 응답 텍스트(가맹점명, 태그, 페르소나 이름 등)를 `innerHTML` 에 그대로 꽂고 있었습니다. 지금 데이터는 우리가 만든 mock 데이터라 실질적 위험은 낮지만, `GEMINI_API_KEY` 가 실제로 쓰이기 시작하면 `ai_tag` 같은 필드가 진짜 LLM 출력이 되므로 방어적으로 `frontend/api.js` 에 `escapeHtml()` 을 추가하고 모든 동적 텍스트에 적용했습니다.
- **예산 도넛 차트 0원 엣지 케이스:** 이번 달 지출이 전부 0원이면 `conic-gradient` 가 마지막 색 하나로 도넛 전체를 채워버리는 CSS 특성이 있어서, `budget.html` 에 0원일 때의 중립 회색 분기를 추가했습니다.
- **정기결제 순차 요청 → 병렬 요청:** `budget.html` 이 카테고리마다 정기결제 예측을 순서대로 `await` 하던 걸 `Promise.allSettled` 로 병렬화했습니다.
- **데모 페르소나 스토리와 실제 리스크 점수 불일치 (내용 버그):** 5명 전원의 실제 `GET /risk/score` 를 확인해보니 이하늘("안정적인 지출"로 적어뒀던 IT 개발자)이 실제로는 CRITICAL(100점, BLOCK) — 유흥업소·금은방·해외 Steam 고액결제로 R001/R002/R003/R005/R007이 동시에 걸린 케이스였습니다. 코드 버그가 아니라 실제 mock 거래 데이터가 그렇게 만들어진 것이라(검증용으로 만든 `/tmp/risk_survey.py` 로 38명 시드 유저 전체를 리플레이해봐도 거래 이력이 있는 유저 중 가장 낮은 점수도 HIGH(91점)였습니다 — 이 mock 데이터셋엔 "위험 신호가 전혀 없는" 유저 자체가 없습니다), 이하늘의 스토리를 "이상거래 확인이 필요한 케이스"로 다시 쓰고 김민철("특별한 위험 신호 없는" → "애매한 결제 한두 건 빼면")도 같이 완화했습니다. 이제 5명이 실제 점수(MEDIUM~CRITICAL)와 태그라인이 서로 모순되지 않습니다.
- **`models/*.pk1` 이 `.gitignore` 에 걸려있던 배포 버그 (심각):** Dockerfile은 `COPY models ./models` 로 학습된 Isolation Forest 모델(`models/anomaly_iforest.pk1`, 1.7MB)을 배포 이미지에 넣는데, 이 파일 자체가 깃에 안 올라가게 막혀 있었습니다. 서버가 죽진 않습니다 — `app/anomaly_model.py` 가 모델 파일이 없으면 이상탐지 점수를 조용히 항상 0.0으로 반환하도록 설계돼 있어서요. 하지만 이 모델이 잡도록 만든 위치급변/신규기기 이상거래 (룰 엔진으로는 애초에 못 잡는 유형, 정답지 기준 전체 이상거래의 상당수)를 배포 환경에서만 하나도 못 잡는, 걸론 티가 안 나는 반쪽짜리 AI 이상탐지가 되는 셈이라 실제로 배포하기 전에 잡아야 했습니다. `.gitignore` 에서 `models/*.pk1` 을 빼고(`!models/*.pk1`), 실제로 `git add` 했을 때 이 파일이 스테이징되는 것까지 확인했습니다.
- **에러 처리/엣지 케이스 점검:** 잘못된 형식의 `user_id`, 존재하지 않는(형식은 맞는) `user_id`, 누락된 쿼리 파라미터를 각 API에 직접 요청해봤습니다 — 전부 500 에러나 스택 트레이스 없이

깔끔한 4xx/빈 배열/기본값으로 처리되고, 프론트도 `errorHtml()` 로 화면이 깨지지 않고 에러 메시지를 보여줍니다.

- **scikit-learn 버전 불일치 (실제로 배포 환경을 흉내내서 찾음, 심각):** "확실하냐"는 질문을 받고, 지금까지 쓰던 개발 환경 말고 `requirements.txt` 만으로 완전히 새 가상환경을 만들어서 `Dockerfile`의 `copy` 순서 그대로 재현해봤습니다. 그랬더니 `models/anomaly_iforest.pkl` 을 로드할 때 `scikit-learn`이 "이 모델은 1.8.0으로 저장됐는데 지금 1.5.2로 불러오는 중이라 결과가 틀릴 수 있다"는 경고를 냈습니다 — 지금까지 로컬에서 테스트해온 서버는 이 저장소용으로 새로 만든 환경이 아니라, 이미 `scikit-learn 1.8.0`이 깔려 있던 개발 환경이었어서 이 문제가 안 보였던 것입니다. 실제로 모델을 학습시킨 버전(1.8.0)에 맞춰 `requirements.txt` 를 고쳤고, 이 값으로 정말 처음부터 새 가상환경을 만들어서 경고 없이 모델이 로드되는 것과 이하늘 점수가 기존과 동일하게 100/CRITICAL로 나오는 것(모델 결과 자체는 안 바뀜)까지 확인했습니다.
- **테스트 중 로컬 PostgreSQL/Redis가 내려가 있던 걸 발견:** 위 재현 과정에서 서비스가 죽어있는 걸 보고 다시 살렸습니다 — 코드 문제는 아니고 로컬 검증 환경 문제였지만, 서비스가 죽으면 API가 500을 낸다는 것 자체는 다시 확인한 셈입니다.
- **risk.html 에 escapeHtml 적용이 빠진 곳 1개 발견:** 심각도 칩(`sev`, 예: "CRITICAL")이 다른 필드들과 다르게 그대로 `innerHTML` 에 꽂히고 있었습니다. `sev` 는 지금은 백엔드가 LOW/MEDIUM/HIGH/CRITICAL 중 하나로만 내려주는 값이라 실질 위험은 낮지만, "전부 다 `escapeHtml` 처리했다"는 말이 정확하지 않았던 부분이라 마저 고쳤습니다.
- 위 항목들 반영 후 Playwright 재렌더링(콘솔 에러 없음)과 `python -m pytest tests/` 재실행 모두 다시 통과 확인했습니다.

7. 다음에 할 일

1. ~~mock_transactions.csv를 transactions 테이블에 넣는 스크립트~~ → 완료 (`data_pipeline/ingestion.py`)
2. ~~카테고리 매핑~~ → 완료 (`data_pipeline/categorizer.py`, ingestion 시 자동 적용)
3. ~~user_risk_profiles 실제 계산~~ → 완료 (`scripts/build_user_profiles.py`)
4. ~~예산통제-Gig-Score-절세장부 API 뼈대~~ → 완료 (8/26, 6-1번 참고)
5. ~~AI 절세장부 실제 로직 연동~~ → 완료 (8/28, 조소현님 설계 + 6-2번 참고)
6. ~~Isolation Forest 모델 + 룰 엔진 앙상블~~ → 완료 (8/28, 6-3번 참고)
7. **(우선순위 높음, 아직 미해결) daily_limit 산식 재조정** — 6-3번에서 발견: 지금 산식이 너무 타이트해서 R002 오탐이 많습니다(정상 거래의 상당수가 걸림). `scripts/build_user_profiles.py` 의 `daily_limit = max(avg_amt * avg_daily_txns * 3, single_limit * 1.2)` 에서 배수(3배)를 완화하거나, `single_limit`처럼 p95 기반으로 바꾸는 걸 추천합니다.
8. ~~정기결제 감지 + 예산 알림 트리거~~ → 완료 (8/30, 조소현님 설계 + 6-4번 참고)
9. (은검) 건보료 위험 경고 로직 + 소득 예측
10. ~~(은검) Gig-Score 산식을 임시 버전에서 실제 지표로 교체~~ → 완료 (9/2, 아래 "8. 9/2 업데이트" 참고)
11. ~~디자인 시안(민트-그린 헬스핏) 3개 화면~~ → 완료 (8/30, 6-5번 참고). 다음: 실제 API 데이터로 연동
12. ~~보안 점검 및 수정 (.gitignore, API 키 인증, CORS, DB 비밀번호)~~ → 완료 (8/30, 6-5번 참고)
13. ~~프론트엔드-백엔드 실연동 (데모 모드, 5개 페르소나 × 3화면)~~ → 완료 (8/30, 6-6번 참고). 사용자 컨펌 대기 중

14. 비전공자용 로컬 실행 가이드 작성

15. ~~(9/2 발견) 실제 배포 시 DB가 완전히 빈 상태로 뜨는 문제~~ → 완료 (9/2, "9. 9/2 두 번째 업데이트" 참고 — 앱이 시작할 때 스스로 스키마/데모 데이터를 채우도록 고침). 실제 배포(공모전 MVP는 깃허브 링크가 아니라 **9/7 10:00까지 살아있는 배포 URL**이 필요 — daker.ai 규정 확인함) + 깃허브 리포 정리는 아직 남아있음

16. (선택) R006_단기반복 처럼 "몇 번째부터 걸리는지"가 헛갈릴 수 있는 규칙은 데모 시나리오 대본에서 미리 설명 문구를 준비해두면 좋습니다.

8. 9/2 업데이트 — "N잡러 특화" 강화 + 배포 전 치명적 버그 발견

리스크방어/예산통제가 사실 N잡러 전용 기능이 아니라 어느 은행 앱에나 있는 일반적인 기능이라, 서비스가 "그냥 금융 앱"처럼 보인다는 문제의식에서 출발했습니다.

1) 수입 플랫폼 연결 기능 (신규) — `app/routers/income.py`, `frontend/home.html`, `frontend/connect.html`

N잡러의 핵심 페인포인트("여러 플랫폼에서 수입이 들쭉날쭉 들어와서 정신없다")를 정면으로 다루는 기능이 지금까지 하나도 없었습니다. 실제 오픈뱅킹/마이데이터 연동은 이번 MVP 범위 밖이라, "계좌/플랫폼을 연결하면 그 수입이 보인다"는 흐름을 더미 데이터로 재연했습니다: `income_sources / income_events` 테이블(`sql/init.sql`)을 새로 만들고, `scripts/seed_demo_income.py` 로 페르소나별 2~3개 수입 플랫폼(배달/프리랜서/콘텐츠/본업)과 정산 내역을 채웠습니다. 페르소나마다 플랫폼 하나는 일부러 미연결 상태로 남겨둬서, `connect.html` 에서 "연결하기"를 실제로 눌러볼 수 있게 했습니다 — 누르면 로딩 연출 후 "OO 연결 완료! 최근 N건의 정산 내역을 찾았어요" 토스트가 뜨면서 이미 있던 데이터가 드러납니다.

2) "한 장 요약" 홈 화면 (신규) — `frontend/home.html`

지금까지는 예산/리스크/절세장부가 각각 다른 화면이라, 여러 곳을 확인해야 하는 정신없음이 앱 안에서도 그대로 재현되고 있었습니다. `home.html` 을 새 기본 진입 화면으로 만들어서 Gig Score 게이지 + 이번 주 수입 요약 + "오늘 확인할 것" 체크리스트 3가지만 한 화면에 모았습니다. `index.html` 의 페르소나 선택 링크도 `budget.html` 이 아니라 `home.html` 로 바뀌었고, 모든 화면의 하단 네비게이션에 "홈" 탭이 추가됐습니다.

3) Gig Score 실제 산식 완성 — `app/routers/gig_score.py`

8/26부터 "8/30-31 교체 예정"이라고만 적혀있던 스텝이 끝까지 방치돼 있었습니다(거래 건수/지출안정성 두 가지만 보는 임시 산식). 위 수입 플랫폼 연결 기능이 생기면서, 설계서 취지에 맞게 (a) 플랫폼 연결 성실도, (b) 지출 안정성(기존 유지), (c) 업무경비 인정률 (조소현님의 AI 절세장부 분류 로직 재사용) 세 지표로 교체했습니다.

4) (심각, 실제로 재현해서 찾음) "이번 달 지출"이 실서버 시각 기준이라 심사 기간 내내

0원으로 보이는 버그 — `app/demo_clock.py` (신규), `app/redis_client.py`, `app/routers/budgets.py`, `scripts/seed_demo_personas.py`

mock 거래 데이터는 2026-08-22에 끝나는데, 예산 사용량 계산(`budgets.py`)과 Redis 캐시 키(`redis_client.py`)가 전부 실서버 시각(`date.today()`)으로 "이번 달"을 판단하고 있었습니다. 9/2에 `scripts/seed_demo_personas.py` 를 다시 돌려서 직접 재현해보니 5명 전원 "이번 달 실지출 합계 0원"이 나왔습니다 — 즉 실제 달력이 9월로 넘어가는 순간(하필 심사 기간인 9/7~9/11 내내), 그렇게 공들여 검증한 예산통제 화면이 심사위원 눈앞에서 전부 0%/미사용으로 보이는 심각한 문제였습니다. "이번 달"을 실서버 시각이 아니라 이 mock 데이터셋의 가장 최근 거래 시각(`get_demo_now()`, 8/22)으로 고정하도록 고쳐서, 심사 기간 중 어느 날 열어봐도 항상 같은(올바른) 결과가 나오게 했습니다.

5) (배포 전 반드시 확인, 아직 미해결) DB가 빈 상태로 배포될 위험 — 위 "7. 다음에 할 일"의 15번 항목 참고. 지금 이대로 새 서버에 배포하면 5개 페르소나 데모가 전부 빈 화면으로 나옵니다.

위 1~3번 반영 후 Playwright로 홈/연결 화면을 실제 클릭까지 해보며 콘솔 에러 없음을 확인했고(구글 폰트 preconnect 실패와 favicon.ico 404는 이 검증 샌드박스의 네트워크 제한 때문이며 실제 배포 환경에서는 발생하지 않음 — 무해함), 기존 예산/리스크/절세장부 화면도 네비게이션 변경 후 다시 스크린샷으로 확인했습니다.

9. 9/2 두 번째 업데이트 — 배포 자동화(DB 자동 시딩) + 보안 점검

"배포만 하면 되게 완벽하게 만들어달라"는 요청을 받고, 위 "8. 9/2 업데이트"의 5번(DB가 빈 상태로 배포될 위험)을 실제로 고치고, API/보안도 한 번 더 점검했습니다.

1) DB 자동 시딩 (신규) — app/startup_seed.py, app/main.py, Dockerfile

앱이 시작할 때 스스로 (a) sql/init.sql 을 실행해서 스키마를 만들고(이미 있어도 안전 — 전부 IF NOT EXISTS / ON CONFLICT 라서 매번 실행해도 됩니다), (b) transactions 등 각 테이블이 비어있는지 확인해서 비어있는 단계만 data_pipeline/ingestion.py → scripts/build_user_profiles.py → scripts/seed_demo_personas.py → scripts/seed_demo_income.py 순서로 자동 실행하도록 만들었습니다. 이 스크립트들이 있는 data/·scripts/·sql/ 폴더를 Dockerfile 에 추가했습니다(전부 합쳐 1MB 미만이라 이미지 용량에 영향 거의 없음).

실제로 완전히 빈 Postgres DB + 빈 Redis에 연결한 상태로 처음부터 서버를 띄워서 검증했습니다: 스키마 생성 → 4,978건 거래 적재 → 35명 리스크 프로파일 계산 → 5명 페르소나 예산/리스크 이벤트 → 15개 수입 플랫폼까지 전부 자동으로 채워지고, 15초 안에 /health 가 정상 응답하는 것까지 확인했습니다. 그 상태에서 서버를 재시작해도(=이미 데이터가 있는 상태) 다시 채우지 않고 즉시 뜨는 것, 데이터가 중복 생성되지 않는 것도 확인했습니다. 즉 **어떤 Postgres/Redis에 연결해주든(관리형 호스팅 포함) 이 이미지를 docker build + 실행만 하면 5개 페르소나 데모가 바로 동작합니다** — 배포 후 사람에 따라 스크립트를 돌릴 필요가 없습니다.

2) 보안 점검

- docker-compose.yml 의 Postgres(5432)/Redis(6379) 포트가 모든 인터페이스에 열려 있던 것을 127.0.0.1 에만 묶도록 고쳤습니다. 이 파일 그대로 공인 IP가 있는 서버(VPS 등)에 배포하면, 특히 Redis는 기본적으로 비밀번호 인증이 아예 없어서 인터넷 누구나 접속할 수 있는 상태였습니다. api 컨테이너는 도커 내부망(db:5432, redis:6379)으로 접속하므로 이 변경으로 앱 동작에는 영향이 없습니다 — 외부 접근만 막힙니다. (Railway·Render처럼 컨테이너 하나만 빌드해서 관리형 DB에 연결하는 방식으로 배포한다면 이 docker-compose.yml 은 애초에 안 쓰이니 해당 없음.)
- 하드코딩된 API 키/토큰/자격증명이 있는지 코드 전체를 검색했습니다 — .env / .env.example 을 제외하면 없었고, 두 파일 다 실제 값이 아니라 빈 값/플레이스홀더만 있습니다. .env 는 .gitignore 에 걸려 있어 깃에도, 이 zip에도 포함되지 않습니다.
- app/security.py 의 X-API-Key 게이트(8/30 추가)는 그대로 뒀습니다 — API_ACCESS_KEY 를 비워두면 인증 없이 열려있는 상태인데, 일부러 키를 안 넣었습니다. 이 서비스는 로그인에 아예 없는 공개 데모 모드(데이콘 운영진이 명시적으로 허용한 방식)라서 데이터 자체가 전부 가짜이고, 프론트(frontend/api.js)에 넣는 키는 누구나 페이지 소스에서 볼 수 있어 실질적인 보안 효과가 거의 없습니다. 오히려 배포 시 서버 쪽 API_ACCESS_KEY 와 프론트 쪽 API_KEY 를 정확히 맞추지 않으면 전체 데모가 깨지는 위험이 더 커서, 이번

배포에서는 고는 쪽을 기본값으로 유지했습니다. 만약 이 API를 정말 잠그고 싶다면

`.env` 에 `API_ACCESS_KEY` 값을 넣고 `frontend/api.js` 의 `API_KEY` 상수에 **정확히**

같은 값을 넣어야 합니다(하나만 바꾸면 전체가 401로 막힙니다).

- SQL 인젝션: 모든 쿼리가 SQLAlchemy `text()` + 파라미터 바인딩(`:uid` 등)이라 문자열 조합으로 값을 끼워넣는 곳이 없습니다(이번에 추가한 `income.py` 도 동일한 패턴).

- 에러 응답이 스택 트레이스나 내부 경로를 노출하지 않는지도 다시 확인했습니다(8/30에 이미 통일된 `{"error": {...}}` 포맷 그대로 유지).

10. 9/2 세 번째 업데이트 — 보안 강화 집중 점검 ("금융이라 보안이 제일 중요해" 요청)

금융보안원이 주최하는 공모전이라는 점을 감안해서, 지금까지 넣은 보안 조치를 정리하고 더 보강할 곳을 찾아 추가로 손봤습니다. 순서대로: (A) 지금까지 이미 있던 것, (B) 이번에 새로 찾아서 고친 것, (C) 의도적으로 남겨둔 한계.

A. 이미 있던 보안 조치 (요약)

- SQL 인젝션 방지: 전 쿼리 SQLAlchemy `text()` + 바인드 파라미터 (`app/security.py` 외 전체 라우터, 6-5-9번 참고)
- 공유 키 기반 API 접근 제어(`X-API-Key`, `app/security.py`) — 배포 시 켤 수 있음
- CORS 화이트리스트(* 금지)
- 에러 응답 통일 포맷(스택 트레이스/내부 경로 미노출)
- `.env` (비밀번호/키) `.gitignore` 처리, 하드코딩된 비밀번호 없음(9번에서 전체 재확인)
- `docker-compose.yml` 의 Postgres/Redis 포트를 `127.0.0.1` 에만 바인딩(9번)

B. 이번에 새로 찾아서 고친 것

1) 의존성 취약점 (pip-audit로 발견, 35건)

`pip-audit -r requirements.txt` 를 돌려서 처음으로 전체 의존성을 스캔해봤습니다. 결과:

패키지	이전	이후	취약점 수
python-dotenv	1.0.1	1.2.3	1건
pillow (전이 의존성, google-genai 가 끌고 옴)	11.3.0 (핀 안 됨)	12.3.0 (직접 핀)	18건
starlette (전이 의존성, fastapi 가 끌고 옴)	0.38.6	1.6.0	7건

`fastapi` 를 0.115.0 → 0.141.1로, `starlette` 를 직접 1.6.0으로 편했습니다(자세한 이유는 `requirements.txt` 상단 주석 참고 — 요약하면 `fastapi` 0.121.0만 해도 `pydantic` 2.9.2와 그대로 호환돼서 더 안 올러도 됐지만, `starlette` 취약점 중 일부는 1.1.0~1.3.1까지 요구해서 `starlette`를 별도로 최신까지 올렸습니다). `google-genai` 는 0.3.0 → 1.28.0으로 올렸는데, 기존 0.3.0이 `pillow<12.0.0` 을 강제해서 안전한 `pillow` 버전과 충돌했던 게 이 업그레이드로 해결됩니다(1.x부터는 `pillow`가 필수 의존성에서 아예 빠짐). `data_pipeline/expense_classifier.py` 가 쓰는 `genai.Client(...)` / `client.models.generate_content(...)` API 형태는 이 범위에서 그대로 동작하고, 애초에 이 함수 전체가 `try/except` 로 감싸여 있어 API 응답 형태가 조금 달라져도 규칙 기반 폴백으로 안전하게 넘어갑니다.

검증: 새 `requirements.txt` 로 완전히 새 가상환경(`python -m venv`)을 만들어

`pip install -r requirements.txt` → `pip check` (의존성 충돌 없음 확인) →

pip-audit (0건 확인, 아래 스캔 결과 참고) → 완전히 빈 Postgres DB + 빈 Redis에 연결해서 서버 기동(스키마 생성 + 5명 페르소나 데모 데이터 전부 자동 시딩까지 정상 완료) → gig-score/income(요약·목록·연결하기)/budgets/risk 등 전 API 엔드포인트를 curl로 직접 호출해서 응답 확인까지 전부 다시 통과시켰습니다.

```
Found 9 known vulnerabilities in 2 packages # pip, setuptools (빌드 툴체인 자체 -
# requirements.txt에 없고 배포되는
# 앱 코드와 무관)
```

2) 보안 응답 헤더 미들웨어 (신규) — app/security_headers.py

Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, (HTTPS일 때) Strict-Transport-Security를 모든 응답에 붙입니다.

CSP는 frontend/*.html 이 페이지마다 인라인 <script> 블록을 쓰고 있어서

script-src 'self' 'unsafe-inline' 로 뒀습니다 — 완벽하진 않지만(다음 단계로 인라인 스크립트를 전부 별도 .js로 옮기고 nonce 기반으로 강화할 수 있음), 외부 도메인에서 스크립트를 끌어오는 가장 흔한 XSS 공격 형태는 확실히 막습니다. Playwright로 6개 페이지를 전부 다시 열어서 CSP 때문에 깨지는 게 없는지(콘솔에 "Refused to..." 같은 CSP 위반 로그가 없는지, 인라인 스크립트-구글 폰트 스타일시트가 정상 동작하는지) 확인했습니다.

3) Rate Limiting (신규) — app/rate_limit.py

IP당 분당 120회로 /api/v1/* 요청을 제한합니다(정적 프론트 파일은 제외 — 페이지 하나 로드에도 여러 파일을 받아오니까요). 무차별 스크래이핑이나 API_ACCESS_KEY 무차별 대입 시도를 최소한이라도 막기 위함입니다. 지금은 단일 프로세스 배포(unicorn 워커 1개)라 메모리 기반 카운터로 충분합니다 — 나중에 워커를 여러 개로 늘리면 Redis 기반 공유 카운터로 바뀌어야 합니다(코드 주석에 남겨둠). 실제로 130회 연속 요청을 보내서 121번째부터 429 가 나오는 것, 429 응답에도 보안 헤더가 정상적으로 붙는 것, Retry-After 헤더가 붙는 것까지 확인했습니다.

4) Redis 비밀번호 인증 (신규) — docker-compose.yml, .env / .env.example

포트를 127.0.0.1 로 막아도(9번) "도커 내부망에 컨테이너 하나만 뚫리면 Redis는 완전 무방비"라는 문제가 남아있어서, --requirepass 로 인증을 추가했습니다. .env 의 REDIS_PASSWORD 를 redis 서비스의 command 와 헬스체크에 그대로 씁니다. REDIS_URL 을 redis://:<비밀번호>@redis:6379/0 형식으로 바꾸면 redis-py (app/redis_client.py , app/database.py)가 URL에서 비밀번호를 읽어 자동으로 인증하므로 **앱 코드는 수정하지 않았습니다** — 실제로 로컬에 --requirepass 를 켜 별도 Redis 인스턴스를 띄우고 이 URL 형식으로 연결해서 PING 부터 budgets/usage API 호출까지 정상 동작하는 것을 직접 확인했습니다.

5) 컨테이너 비루트 실행 (신규) — Dockerfile

지금까지 컨테이너 안에서 앱이 root로 돌고 있었습니다. 앱이 런타임에 /app 아래 아무 파일도 쓰지 않는 걸 확인했고(모델 학습 스크립트(app/anomaly_model.py 의 train())는 개발 중 수동으로만 돌리는 오프라인 스크립트라 컨테이너 안에서 실행되지 않음), 전용 비루트 유저(appuser)로 내려도 기능에 영향이 없어서 적용했습니다. 만에 하나 앱 코드에 원격 코드 실행으로 이어지는 취약점이 있어도, root 권한으로 컨테이너 전체를 장악하는 것보다는 피해 범위가 훨씬 줄어듭니다.

(참고: 이 샌드박스 환경은 Docker 데몬이 실제로 뜨지 않아서 docker build / docker run 자체로 이 Dockerfile을 끝까지 실행해보진 못했습니다. 대신 실제 배포 이미지와 동일한 requirements.txt 로 완전히 새 가상환경을 만들어 위 B-1의 전체 시나리오를 검증했고,

Dockerfile 변경 자체는 표준적인 `useradd + chown + USER` 패턴이라 위험도가 낮다고 판단했습니다. 배포 담당자가 `docker build` 로 이미지를 만드는 시점에 한 번 더 정상 기동을 확인해주시면 가장 확실합니다.)

C. 의도적으로 남겨둔 한계 (강조해서 남겨둡니다)

- **IDOR(다른 사람 데이터 조회) 미차단:** `user_id` 를 쿼리 파라미터로 그대로 받기 때문에, API를 호출할 수 있는 사람은(= `API_ACCESS_KEY` 를 알거나, 지금처럼 꺼둔 상태라면 누구나) `user_id` 만 바뀌어서 다른 사람의 예산/리스크/절세장부/수입 데이터를 조회할 수 있습니다. 이건 진짜 로그인·세션 기반 인가(authorization)가 있어야 막을 수 있는데, 이번 데모는 **로그인이 아예 없는 공개 데모 모드**(데이콘 운영진이 명시적으로 허용한 방식)이고 데이터 자체가 5개 페르소나 + 일반유저 모두 처음부터 끝까지 가짜(mock)라서, 실제 피해로 이어지는 개인정보 유출이 아닙니다. **다만 이 구조를 실제 서비스(진짜 계좌·진짜 사용자)로 그대로 확장한다면 가장 먼저 넣어야 하는 게 바로 이 부분입니다** — 세션 기반 로그인을 붙이고, 모든 API가 "요청자 본인의 `user_id`"만 조회하도록 서버에서 강제해야 합니다.
- `API_ACCESS_KEY`는 여전히 기본값이 꺼진 상태입니다(9번에서 설명한 이유 그대로 — 프론트 JS에 키가 그대로 노출되는 구조라 실질적 방어 효과가 적고, 서버/프론트 키 불일치로 데모 전체가 깨질 위험이 더 큼). 켜고 싶다면 `.env` 의 `API_ACCESS_KEY` 와 `frontend/index.html` 상단의 `API_KEY` 상수(11번 참고, 9/5부터 `frontend/api.js` 가 아니라 이 파일로 옮겨졌습니다)를 정확히 같은 값으로 맞추면 됩니다.
- Rate limiting은 단일 프로세스 전제입니다(위 B-3 참고) — 멀티 워커로 배포를 바꾸면 Redis 기반으로 교체해야 합니다.
- CSP의 `script-src` 에 '`unsafe-inline`' 이 남아있습니다(위 B-2 참고) — 완전히 없애려면 프론트 인라인 스크립트를 전부 별도 `.js` 파일로 분리하는 리팩터링이 필요합니다.

11. 9/5 업데이트 — 프론트엔드 디자인 전면 교체 (하은님 시안 적용)

은겸님이 하은님의 새 시안(`sidegig_ui_3.html`, `frontend/index.html` 단일 파일)으로 가기로 결정해서, 기존에 우리가 붙여왔던 화면(홈/예산/리스크/절세장부/연결, `home.html`, `shield.html` 등)을 전부 이 파일 하나로 교체했습니다. 마감(9/7 10:00)이 임박한 만큼 "디자인 파일만 바꿔치기"에 그치지 않고, 실제로 붙여서 콘솔 에러 없이 뜨는지까지 검증했습니다.

1) 파일 교체 (하은님 README_프론트_인수인계.md 안내대로)

기존 `frontend/*.html`, `api.js`, `shared.css` (9/2에 만든 Risk Shield 포함)를 전부 `frontend/legacy/` 로 옮기고, 하은님이 준 `index.html` 을 새 `frontend/index.html` 로 넣었습니다. `app/main.py` 의 정적 파일 마운트 설정은 손댈 게 없었습니다(하은님이 이미 지금 구조에 맞춰서 확인해두심).

2) `USE_API`를 켜고, 실제로 붙는 걸 확인 — 6곳 라이브 연동

하은님 파일은 `USE_API = false` (전부 목데이터)로 넘어왔습니다. 백엔드에 이미 있는 엔드포인트와 실제로 맞춰보니 아래 6개는 지금 그대로 붙습니다: `budgets/usage`, `budgets/forecast` (카테고리별로 불러서 프론트에서 합산), `budgets` 의 정기결제 감지 결과, `risk/score/{user_id}`, `risk/events`, `expense/classify`. `USE_API = true` 로 바꾸고 완전히 빈 Redis 상태에서부터 다시 켜서(`scripts/seed_demo_personas.py` 로

캐시까지 재적재) 예산 도넛/월말 예측/리스크 게이지/이상거래 목록/절세장부 분류 카드가 전부 실제 DB 값으로 뜨는 것까지 확인했습니다.

나머지(수입 요약·플랫폼 연결·맞춤 상품·건보료·세금 일정·내 장부·24시간 타임라인)는 하은님이 이미 README에 우선순위까지 정리해둔 대로 아직 백엔드 API가 없어서 목데이터로 남아있습니다 — 지금 급하게 다 만들지 않고, 하은님이 적어둔 그 목록과 우선순위를 그대로 다음 할 일로 남겨둡니다.

3) 실연동하다 실제로 찾은 버그 2건 (하은님 파일 잘못이 아니라 우리 쪽 배선 문제)

- **CSP가 새 폰트 CDN을 막고 있었음** — `app/security_headers.py` 의 CSP가 `fonts.googleapis.com` 만 허용해서, 하은님 파일이 쓰는 Pretendard 폰트 (`cdn.jsdelivr.net`)를 브라우저가 조용히 차단하고 있었습니다(화면이 깨지지 않고 시스템 폰트로 자동 대체되어서 눈으로는 티가 안 남 — 콘솔에만 에러가 찍힘). `style-src` `font-src` 에 `https://cdn.jsdelivr.net` 을 추가해서 고쳤습니다.
- **리스크 점수 산출근거가 전부 "+0"으로 보임** — `risk/events` 가 실제로는 이벤트별 점수(`score` 컬럼)를 내려주는데, 프론트 어댑터가 그 값을 `pts` 필드로 옮겨담지 않아서 "산출근거" 화면의 막대그래프/칩이 전부 0점으로 표시됐습니다. 어댑터 한 줄(`pts:Math.round(r.score||0)`)을 추가해서 실제 점수가 보이도록 고쳤습니다.
- **데모 유저 ID도 손봤습니다** — 하은님 파일에 박혀있던 `USER_ID` (8/30 기준 김민철)는 그 뒤 시드 데이터가 다시 만들어지며 이미 사라진 UUID였습니다. `app/demo_personas.py` 의 현재 김민철 UUID로 교체했습니다.

검증: Playwright로 수입/예산/리스크/절세장부 4개 탭 전부, 그리고 심사 시연용 "데모 시나리오" 패널의 4개 버튼(정상 거래/이상 거래 차단/예산 초과/초과 예측)까지 직접 클릭해서 콘솔 에러 없음을 확인했습니다(유일하게 뜨는 `/favicon.ico 404`는 파비콘 파일이 없어서 나는 무해한 것 — 8~10번에서부터 계속 있던 것과 동일).

참고해두실 것

- 하은님 파일은 URL 쿼리로 페르소나를 바꾸는 기존 방식이 아니라, 파일 상단 `USER_ID` 상수 하나로 고정된 1인 데모입니다(대신 화면 오른쪽 "데모 시나리오" 패널로 시연 흐름을 보여주는 방식). 심사 때 5개 페르소나를 전부 보여줘야 한다면 페르소나 선택 화면을 다시 붙이는 작업이 추가로 필요합니다 — 지금은 하지 않았습니다.
- 9/2~9/3에 만든 Risk Shield/동적 파킹 기능(`app/cashflow.py`, `app/routers/shield.py`, `shield.html`)은 백엔드 API(`/api/v1/shield/*`)로는 그대로 남아있지만, 이번 새 화면에는 그 개념이 없어서 화면에서는 보이지 않습니다. 하은님 화면의 "세금 통장"(수입 탭)이 비슷한 역할(세금 미리 떼어두기)을 이미 다른 방식으로 담고 있어서, 굳이 두 기능을 역지로 합치지 않았습니다 — 필요하시면 다음 작업으로 논의해주세요.

12. 9/5 대개편 — 배포·신뢰도·화면 전면 정비

배포 링크: <https://nfinity-ai-o2ok.onrender.com> (Render 무료 플랜, GitHub main에 push하면 자동 재배포)
저장소: <https://github.com/graymango7/Nfinity>

12-1. 실제로 틀린 숫자를 내보내고 있던 것 4가지 (전부 수정 완료)

배포 후 5명 페르소나의 API 응답을 전수 확인하다가 찾았습니다. 화면에는 그럴듯하게 보이지만 숫자 자체가 틀렸거나 기능이 죽어 있던 것들입니다.

	증상	원인	조치
Risk Shield 잔액	5명 중 3명이 마이너스 수백만원	수입은 "연결된 플랫폼"만 집계하는데(의도된 UX) 지출은 전부 집계 → 구조적으로 음수	시작 잔액을 <code>user_settings.current_balance</code> 로 명시(오픈뱅킹 자리), 페르소나별 시드
45일 예측	전원 "1일 후 잔고 부족"	위 잔액이 이미 음수라 첫날부터 안전선 아래 = 예측이 아니라 이미 파산	잔액 앵커 수정 후 박지수 9/26(35일 후), 이하늘 9/1, 김민철 부족 없음으로 정상화
정기결제 투사	45일 시뮬레이션에 지출 이벤트 0건 (그래프가 직선)	25~35일 간격만 정기결제로 인정 → 주간·격주 결제 전부 누락	중앙값 기준 규칙성 판정(5~40일, 모든 간격이 중앙값 $\pm 25\%$)로 교체
Gig Score	5명 전원 <code>spending_stability</code> 정확히 0.0 , 총점 293~433	<code>1-min(cv,1)</code> 산식인데 거래 금액은 편차가 커서 cv가 항상 1 초과	<code>1/(1+cv)</code> 로 교체 → 383~529, 지표별 변별력 생김

12-2. 세금·건강보험료를 실제 산식으로 (`app/tax.py` , `GET /api/v1/tax/estimate/{user_id}`)

기존에는 "월평균 수입 $\times 8\%$ (세금), $\times 7\%$ (건보료)"였고, 코드 주석에도 근거가 "프리랜서에게 흔히 권장되는 관행"이라고만 적혀 있었습니다. 이 서비스의 핵심 주장이 "1~2년 뒤 맞을 세금·건보료를 지금 알려준다"인 이상 그 숫자에 근거가 있어야 해서 실제 제도로 교체했습니다.

- **종합소득세**: 단순경비율(업종별) → 기본공제 150만원 → 과세표준 → **8단계 누진세율표**
→ 지방소득세 10% → **기납부 원천징수 3.3% 차감**. 결과가 음수면 환급액입니다.
- **건강보험료**: 2026년 요율 **7.19%** + 장기요양보험료(건보료의 12.95%). 직장가입자는 급여 외 소득 **2,000만원 초과분**에만 소득월액보험료(전액 본인 부담), 지역가입자는 소득 전체.

실제 출력 (배포 서버 기준):

페르소나	5월 종합소득세	건강보험료	경고
김민철(배달 투잡)	36.8만원 환급	월 0원	"앞으로 612만원 더 벌면 2,000만원 기준을 넘습니다"
박지수(프리랜서)	18.1만원 환급	월 37.7만원(지역)	"피부양자 자격 기준 초과"
이하늘(개발자+외주)	42.7만원 환급	월 17.8만원(직장)	"초과분 2,629만원은 회사 지원 없이 전액 본인 부담"

발견: 3.3% 원천징수는 단순경비율을 적용하면 대부분 **과납**이라 오히려 환급이 나옵니다. 그래서 화면 문구도 "세금 떼어두세요"가 아니라 "5월에 돌려받아요 / 건보료는 이만큼 나갑니다"로 바꿨습니다. **2,000만원 경계를 넘기 전에 미리 알려주는 것이** 이 기능의 핵심입니다.

한계는 `app/tax.py` 상단 docstring에 명시했습니다(인적공제·세액공제·다른 소득·재산 점수 미반영).

12-3. 프론트엔드 전면 재작성 (`frontend/index.html`)

기존 화면은 폰 목업 형태라 데스크톱으로 심사할 때 제품이 아니라 목업처럼 보였습니다. 토스 디자인 언어(Pretendard, 토스 블루, 18~20px 라운드, 테두리 대신 여백/그림자) 기준으로 다시 만들었고, 구조를 두 화면 + 3단으로 나눴습니다.

1. **사례 선택 화면** — 3명 카드에 각자의 현재 현금흐름 상태(안전/주의/위험)와 부족 예상일을 미리 띄워서, 고르기 전에 무엇을 보게 될지 알 수 있게 했습니다.
2. **대시보드**

- 1단 요약: "9월 26일, 35일 뒤에 잔고가 30만원 아래로 떨어질 것 같아요" + 현재 잔액 / 쓸 수 있는 돈 / 다음 입금
- 2단 할 일: 실제 데이터로 계산한 행동 3개 (플랫폼 연결 버튼, 건보료 경고, 예산 초과 등)
- 3단 탭: 현금흐름 / 수입 / 지출 / 거래 보안

읽기 전용이 아니라 실제로 조작됩니다 — 아래 셋은 전부 서버 상태를 바꾸고 결과를 다시 불러옵니다.

기능	엔드포인트	눌렀을 때 실제로 일어나는 일
거래 위험도 즉시 검사	POST /api/v1/risk/assess	룰 엔진 7종 + Isolation Forest 앙상블이 판정하고 risk_events 에 기록 → 옆 탐지 이력이 갱신
수입 플랫폼 연결	POST /api/v1/income/sources/{id}/connect	연결된 플랫폼만 예측에 반영되므로 45일 곡선과 세금 추정 이 실제로 바뀐
최소 안전잔액 변경	PUT /api/v1/shield/{id}/settings	저장 후 시뮬레이션 재계산 → 부족 예상일이 바뀜

12-4. 보안 · 운영

- IDOR 차단 (app/demo_guard.py)**: 기존에는 user_id 만 바꾸면 시드된 38명 전체의 예산·리스크·수입을 꺼낼 수 있었습니다(README 10-C에 "의도적 한계"로 적어뒀던 항목). 이 배포는 데모 3명만 공개하면 되므로, 경로·쿼리·본문의 user_id 가 그 3명이 아니면 403으로 막습니다. 진짜 인가는 아니고(실서비스는 세션 로그인 + 서버측 소유권 검사 필요) 공개 데모의 최소 방어선입니다.
- 자가 핑 (app/keepalive.py)**: Render 무료 플랜은 15분 무요청 시 잠들고 재기동에 50초가 걸립니다. 심사 기간(9/7 11:00~9/11 23:59)에 첫 접속이 흰 화면이면 안 되므로, 앱이 10분마다 자기 /health 를 호출해 깨어 있게 유지합니다. (GitHub Actions 스케줄이 더 정석이지만 사용 중인 토큰에 workflow 권한이 없어 앱 내부 방식으로 했습니다.)
- 페르소나를 5명 → **3명**으로 줄였습니다(김민철·박지수·이하늘). 최유진·정다운 데이터가 DB에 남아 있어도 app/demo_personas.py 목록에 없으면 화면·API 어디에도 노출되지 않습니다.

12-5. 심사용 3분 시연 대본

- 선택 화면** — "N잡러 3명 중 현금흐름이 위태로운 박지수를 봅니다." (카드에 이미 주의, 09/26 잔고 부족 예상 이 보임)
- 요약** — "9월 26일에 잔고가 바닥납니다. 지금 145만원 있지만 세금·건보료 빼면 107만원이고, 다음 입금은 8월 30일입니다." → 은행 앱은 과거 지출을 보여주지, 이걸 못 합니다.
- 현금흐름 탭** — 45일 곡선에서 정산 유입(계단)과 정기 지출을 보여준 뒤, 정산 10일 지연 탭 클릭 → "플랫폼 정산이 밀리면 부족 시점이 앞당겨집니다."
- 세금 카드** — "연 5,573만원 → 필요경비 64% → 과표 1,845만원 → 15% 구간. 이미 3.3%로 184만원을 냈으니 5월에 18만원을 돌려받습니다. 대신 건보료가 월 37.7만원 나가고, 2,000만원 기준을 넘어 피부양자 자격이 사라졌습니다."
- 수입 탭 → 연결하기** — 미연결 플랫폼을 실제로 연결 → 토스트가 뜨고 예측·세금이 재계산됨. "연결할수록 예측이 정확해집니다."
- 거래 보안 탭** → 심야 금은방 고액 프리셋 → 위험도 검사 → **CRITICAL 100점 / 결제 차단**, R001·R002·R003·R005 네 규칙이 동시에 걸린 근거까지 표시. 탐지 이력에도 즉시 추가됨.

12-6. 남은 과제

- 진짜 인가(세션 로그인 + 서버측 소유권 검사) — 지금은 데모 화이트리스트로 대체
- 실제 오픈뱅킹/마이데이터 연동(현재는 사용자가 알려준 잔액을 기준으로 사용)
- `GEMINI_API_KEY` 를 넣으면 업무경비 분류가 실제 LLM 호출로 동작(현재는 규칙 기반 폴백)
- 종합소득세 인적공제·세액공제, 건보료 재산 점수 반영

부록 · 사용자 인터뷰 가이드

N잡러 사용자 인터뷰 가이드

기획서 3장(문제 정의)과 6장(기대 효과)을 "우리 생각"이 아니라 "확인된 사실"로 바꾸기 위한 자료입니다.
3~5명이면 충분합니다. 1인당 20~30분, 통화나 카톡 음성으로도 됩니다.

0. 누구를 만나야 하나

한 유형에 물리면 "그 사람만 그런 것" 소리를 듣습니다. 아래에서 **최소 3유형**을 섞으세요.

유형	예시	왜 필요한가
주 단위 정산형	배달 라이더, 물류 단기, 대리운전	정산이 잦고 금액이 작아 "언제 들어오는지"가 일상 문제
건당 정산형	프리랜서 디자이너·개발자·마케터, 과외	금액이 크고 불규칙 — 현금흐름 위기가 실제로 발생
월 단위 + 부업형	직장인 + 유튜브·블로그·스마트스토어	2,000만원 건보료 경계에 실제로 걸리는 집단

1. 질문지 (순서대로 물어보세요)

원칙: **우리 서비스를 먼저 설명하지 마세요.** 설명하면 상대가 맞장구쳐줍니다.
과거에 실제로 있었던 일만 물어야 답이 신뢰할 수 있습니다.

A. 문제가 실재하는지 (설명 없이)

- 지금 수입이 들어오는 경로가 몇 개인가요? 각각 언제 들어오나요?
- 최근 6개월 중에** 돈이 들어오기 전에 나갈 돈이 먼저 빠져서 곤란했던 적이 있나요? 그때 어떻게 하셨어요?
- 다음 달에 들어올 돈이 대략 얼마인지 아세요? 어떻게 파악하시나요? (머릿속 / 엑셀 / 앱 / 모름)
- 정산이 예정보다 늦어진 적 있나요? 며칠이나요? 그때 무슨 일이 있었나요?

여기서 **확인하려는 것**: "현금흐름 예측"이 진짜 필요한 기능인지.
2번에서 대부분이 "그런 적 없다"고 하면 우리 전제가 틀린 것이므로 기획을 바꿔야 합니다.

B. 세금·건강보험료

- 5월 종합소득세 신고, 어떻게 하고 계세요? (직접 / 세무서 / 삼점삼류 앱 / 안 함)
- 작년 5월에 세금을 더 냈나요, 돌려받았나요? 금액이 예상과 비슷했나요?
- 건강보험료 고지서를 받고 놀란 적 있나요? 언제, 왜였나요?
- (직장 있는 분께) 부업 소득이 연 2,000만원을 넘으면 보험료가 따로 붙는 것 알고 계셨나요?

여기서 **확인하려는 것**: 8번이 핵심입니다. **모른다고 답하는 비율**이 높을수록
"경계를 넘기 전에 미리 알려준다"는 우리 기능의 가치가 커집니다.

C. 지금 쓰는 대안

- 1. 지금 이걸 관리하려고 쓰는 게 있나요? (가계부 앱 / 엑셀 / 은행 앱 / 없음)
- 2. 그게 부족하다고 느낀 순간은 언제였나요?

D. 마지막에만 — 우리 화면 보여주기

- 1. (배포 URL을 열어 보여주며) "이 화면이 무엇을 알려준다고 보이세요?" — 설명하지 말고 먼저 물어보세요.
- 2. 이 중에 실제로 쓰실 것 같은 기능이 있나요? 반대로 쓸모없어 보이는 건요?
- 3. 돈을 낸다면 얼마까지 낼 의향이 있나요? (0원이라고 해도 그대로 기록)

2. 기록 양식 (그대로 복사해서 쓰세요)

[대상 #1] 유형: 배달 라이더 / 경력 1년 6개월 / 수입 경로 3개

- 정산 주기: 배민 주 1회(월), 쿠팡이츠 주 1회(수), 본업 급여 25일
- Q2 (자금 부족 경험):
- Q4 (정산 지연 경험):
- Q6 (작년 5월 세금):
- Q8 (2,000만원 기준 인지): 알고 있었다 / 몰랐다
- Q11 (화면 첫인상, 설명 전): "....."
- Q12 (쓸 것 같은 기능 / 쓸모없는 기능):
- 인상적이었던 말 그대로 한 줄: "....."

3. 기획서에 넣을 때

인터뷰가 끝나면 아래 형태로 정리해 기획서 3장 끝에 넣으세요.
과장하지 말고 들은 대로 쓰는 게 오히려 설득력이 있습니다.

사용자 인터뷰 (2026년 9월, N명)

배달·프리랜서·부업형 N잡러 N명을 대상으로 각 20~30분 인터뷰를 진행했습니다.

- N명 중 M명이 최근 6개월 내 "수입이 들어오기 전에 고정비가 먼저 빠져나가" 곤란했던 경험이 있었습니다.
- 다음 달 예상 수입을 파악하는 방법으로 M명이 "머릿속으로 대충"이라고 답했습니다.
- 부업 소득이 연 2,000만원을 넘으면 건강보험료가 별도로 부과된다는 사실을 M명이 모르고 있었습니다.
- 인용: "○○○" (프리랜서 디자이너, 경력 3년)

주의: 실제로 만난 인원·답변만 적으세요. 발표 심사에서 "몇 명 만나셨고 어떤 분들이었나요"는 반드시 나오는 질문이고, 그 자리에서 답이 흔들리면 나머지 내용의 신뢰도까지 함께 무너집니다. 숫자가 작아도(3명이어도) 실제로 만난 3명이 지어낸 30명보다 강합니다.